Design and Implementation of a Web-Based Competitive Examination Preparation Platform (CrackuEx)

A Project Report

Submitted in partial fulfilment of the requirements for the
award of the degree of

"Master of Computer Applications (MCA)"

(Distance and Online Mode)

By

Amandeep

Registration Number: 22407625451



Centre for Distance and Online Education

LOVELY PROFESSIONAL UNIVERSITY

PHAGWARA, PUNJAB

2026

Nature of Project: Web-based software system | Category: General / Skill-oriented

DECLARATION

I, **Amandeep**, Registration Number **22407625451**, student of the Centre for Distance and Online Education, Lovely Professional University, Phagwara, Punjab, hereby declare that the work done by me on "Design and Implementation of a Web-Based Competitive Examination Preparation Platform (CrackuEx)" is a record of original work for the partial fulfilment of the requirements for the award of the degree of **Master of Computer Applications (MCA)**. The matter embodied in this report has not been submitted in part or in full to any other university or institute for the award of any degree, diploma or certificate.

I further declare that the software described herein was designed and implemented as a web-based academic project. Wherever published literature, official examination notifications, or software documentation have been consulted, they have been cited in the prescribed IEEE format. No copyrighted text has been reproduced verbatim. I understand the University's policy on plagiarism and academic integrity, and I accept that any proven instance of copied content would render this submission liable to cancellation.

Name of the Student: Amandeep

Registration Number: 22407625451

Programme: Master of Computer Applications (MCA)

Signature of the student: 

Dated: 24/10/2026

ACKNOWLEDGEMENT

I wish to place on record my sincere gratitude to Lovely Professional University and to the Centre for Distance and Online Education for providing the academic framework under which this project could be conceived, implemented and documented. The project course of the Master of Computer Applications programme gave me a structured opportunity to convert classroom knowledge of web technologies into a complete, working software system.

I am thankful to the faculty mentors associated with the distance and online mode for their guidance on problem selection, report organisation and the expectation that a project must be both technically sound and socially relevant. Their insistence on original writing, proper referencing and a clear statement of limitations has shaped the tone of this report.

I also acknowledge the public documentation of the Staff Selection Commission and the Railway Recruitment Boards, which was used only as a factual source for examination structure. Open-source communities around React, Express, MongoDB, Redis and Docker made it possible to build on mature tools rather than inventing infrastructure from scratch. Any remaining errors of design or expression are mine alone.

Finally, I thank my family for the time and quiet that a software project of this size requires. Completing CrackuEx as an MCA project has been as much an exercise in disciplined engineering as it has been an exercise in independent learning.

Amandeep

MCA, Centre for Distance and Online Education

Lovely Professional University, Phagwara

ABSTRACT

Competitive recruitment examinations in India, notably those conducted by the Staff Selection Commission (SSC) and the Railway Recruitment Boards (RRB), attract a very large number of candidates each year. Preparation is no longer confined to printed guides. Aspirants expect short, repeatable practice, a place to keep notes, a way to add their own questions, and a truthful picture of which topics are strong or weak. Many existing websites emphasise marketing claims, predicted ranks or unofficial cut-offs. The present project, named CrackuEx, takes a narrower and more honest path: it is a web-based study platform that supports practice, notes, user-contributed questions and progress tracking without inventing a rank or a cut-off.

The system is implemented as a React 19 single-page application talking to an Express 5 API written in TypeScript. MongoDB stores users, syllabus topics, questions, mock attempts and progress. Redis provides rate-limit counters, optional caching and a BullMQ queue for study reminders. Authentication uses a Bearer JSON Web Token with a server-side tokenVersion so that a new login can invalidate an older session. Drill and competition endpoints never send the correct option to the browser; they issue a signed challenge or session token and grade answers on the server. Notes that may contain markup are sanitised with DOMPurify before display. The API records a request identifier on every response so that a failure can be traced in logs without exposing secrets. Docker Compose starts MongoDB, Redis, the API and a worker, each with healthchecks, and the API image runs as a non-root user.

The work maps to Sustainable Development Goal 4 (Quality Education) by offering a free, structured digital path for government-exam preparation, and it supports Goal 8 in so far as those examinations are a documented route to public employment. The report describes the problem, reviews relevant literature on e-learning and web architecture, presents analysis and design, records the implementation, discusses results and limitations, and states a realistic future scope. No claim is made that the platform has been load-tested at the scale of a national consumer website. The claim that is made, and that the source code supports, is that the application is engineered so that a later operator can add replicas, keep secrets out of the image, and reason about failures using structured logs.

Keywords: web-based learning, SSC, RRB, React, Express, MongoDB, Redis, JWT, computer-based practice, Docker, MCA project

TABLE OF CONTENTS

DECLARATION 1
ACKNOWLEDGEMENT 2
ABSTRACT 3
LIST OF TABLES 6
LIST OF FIGURES 7
LIST OF SCHEMES 7
LIST OF SYMBOLS 7
LIST OF ABBREVIATIONS 8
CHAPTER 1 INTRODUCTION 9
1.1 Background of the Study 9
1.2 The Indian Competitive Examination Context 9
1.3 Problem Statement 10
1.4 Need for the Study 10
1.5 Aim of the Project 11
1.6 Objectives 11
1.7 Scope of the Project 12
1.8 Alignment with Sustainable Development Goals 12
1.9 Nature and Type of the Project 13
1.10 Organisation of the Report 13
CHAPTER 2 REVIEW OF LITERATURE 14
2.1 Introduction to the Review 14
2.2 E-Learning and Instructional Design 14
2.3 Mastery, Taxonomy and Motivation 14
2.4 Computer-Based Testing and Practice Design 15
2.5 Gamification 15
2.6 Web Architecture for Student-Scale Products 16
2.7 Security Literature Relevant to the Project 16
2.8 Existing Practice and the Research Gap 16
2.9 Summary of the Review 17
CHAPTER 3 SYSTEM ANALYSIS AND DESIGN 18
3.1 Introduction 18
3.2 Feasibility 18
3.3 Users and Roles 18
3.4 Functional Requirements 19
3.5 Non-Functional Requirements 19
3.6 Logical Architecture 20
3.7 Client Architecture 21
3.8 Data Design 21

3.9 Progress Model 22
3.10 Security Design 22
3.11 Deployment Design 22
CHAPTER 4 IMPLEMENTATION OF THE PROJECT 24
4.1 Introduction 24
4.2 Technology Stack 24
4.3 Backend Module Convention 25
4.4 Authentication and Session Control 25
4.5 Drill Engine 26
4.6 Syllabus, Notes and Topic Tests 27
4.7 Mocks, Performance and Analytics 27
4.8 MCQ Battle 27
4.9 Reminders and the Worker 27
4.10 AI Explanation Path 28
4.11 Administration 28
4.12 Docker, Redis Lifecycle and Shutdown 28
4.13 Errors, Logging and Front-End Hardening 29
4.14 Testing during Implementation 29
CHAPTER 5 RESULTS AND DISCUSSIONS 31
5.1 Introduction 31
5.2 Completed Learner Loop 31
5.3 Operational Results 32
5.4 Test Results 32
5.5 Discussion against Objectives 33
5.6 Discussion against Literature 33
5.7 Limitations 33
5.8 Summary of the Discussion 34
CHAPTER 6 CONCLUSION AND FUTURE SCOPE 35
6.1 Conclusion 35
6.2 Future Scope 35
6.3 Closing Remark 36
PUBLICATION DETAILS 37
ANNEXURES 38
Annexure A — Secrets and Environment 38
Annexure B — Standard Error Envelope 38
Annexure C — Suggested Local Bring-up 38
Annexure D — Cover and Declaration Note 39
REFERENCES 40

LIST OF TABLES

Table No.	Title
Table 1.1	Scope included in the present release and items left out of scope
Table 1.2	Mapping of the project to selected Sustainable Development Goals
Table 3.1	Functional requirements of CrackuEx
Table 3.2	Non-functional requirements and the mechanism used to address each
Table 3.3	Principal MongoDB collections and their role
Table 4.1	Technology stack used in the implementation
Table 4.2	Representative API route groups
Table 4.3	Security controls implemented in the current codebase
Table 5.1	Implemented learner-facing modules and the evidence of completion
Table 5.2	Health endpoint fields and how an operator should read them
Table 5.3	Automated tests executed during this project
Table A.1	Environment variables that must not be committed to source control

LIST OF FIGURES

Figure No.	Title
Figure 3.1	Logical architecture of CrackuEx
Figure 3.2	Request path from the browser to persistence
Figure 3.3	Progress states used after a topic test
Figure 4.1	Authentication and session control
Figure 4.2	Drill challenge and server-side grading
Figure 4.3	Docker Compose runtime (API, worker, MongoDB, Redis)
Figure 5.1	Learner study loop supported by the application

LIST OF SCHEMES

Nil. The present work is a software engineering project. It does not involve chemical reaction schemes or laboratory protocols.

LIST OF SYMBOLS

The report does not introduce a private mathematical notation. Standard software symbols (HTTP status codes, JSON field names and Docker service names) are used in their ordinary sense and are expanded in the list of abbreviations where needed.

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
AOF	Append-Only File (Redis persistence mode)
CBT	Computer-Based Test
CGL	Combined Graduate Level (SSC examination)
CHSL	Combined Higher Secondary Level (SSC examination)
CORS	Cross-Origin Resource Sharing
CPO	Central Police Organisation (SSC examination)
CSP	Content Security Policy
DTO	Data Transfer Object
GD	General Duty (SSC examination)
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
MCQ	Multiple-Choice Question
MCA	Master of Computer Applications
MTS	Multi-Tasking Staff (SSC examination)
OTP	One-Time Password
OWASP	Open Worldwide Application Security Project
REST	Representational State Transfer
RRB	Railway Recruitment Board
SDG	Sustainable Development Goal
SMTP	Simple Mail Transfer Protocol
SPA	Single-Page Application
SSC	Staff Selection Commission
TTL	Time To Live
UI	User Interface

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

Public recruitment in India continues to use large-scale, standardised written examinations as a filter. The Staff Selection Commission notifies graduate, higher secondary and constable-level examinations whose papers are now delivered largely as computer-based tests. Railway Recruitment Boards run a parallel stream of notifications for Group C and related posts. For an aspirant the calendar is unforgiving: a notification, an application window, an admit card, a CBT, and, for some posts, further stages. Between those dates the candidate must practise under time pressure, revise a published syllabus, and decide which topics still leak marks.[6,7]

Digital learning research has already shown that well-designed online practice can match or exceed some classroom-only treatments when feedback is timely and the learner can repeat a task.[1,3] That finding does not automatically bless every coaching website. Many commercial portals mix a small question bank with a large amount of promotional copy. Others display a predicted rank that cannot be justified from the data the user has actually produced. The educational risk is not only wasted money; it is misplaced confidence. A platform that tells a candidate they are "AIR ready" after twenty questions is not an assessment tool. It is advertising.

CrackuEx was therefore framed as an engineering problem rather than a marketing problem. The product question was simple: can a web application give an SSC-first aspirant a daily loop of practice, notes, self-authored questions and topic-level progress without inventing ranks or cut-offs? The academic question sitting under that product question is equally simple: can those features be built with a readable module layout, explicit error contracts, and an operations story (Docker, Redis, health checks, graceful shutdown) that a later engineer can run without folklore?

1.2 The Indian Competitive Examination Context

SSC Combined Graduate Level (CGL) is aimed at graduate-level posts across ministries and attached offices. Combined Higher Secondary Level (CHSL) addresses 10+2 level posts. General Duty (GD) and Central Police Organisation (CPO) examinations serve uniformed recruitment. Multi-Tasking Staff (MTS) covers a different band of Group C

work. These names matter for a software project because each exam has a published pattern, a subject mix and a negative-marking rule that must not be guessed from hearsay. The landing pages of CrackuEx therefore treat official notices as the source of examination facts, and they refuse to mint a cut-off that the Commission has not published.[6]

Railway examinations overlap with SSC in quantitative aptitude, reasoning and a portion of general awareness, but they are not the same product. A respectful design lets a user pick an exam, then see syllabus-shaped practice for that exam, rather than dumping a generic "government job" quiz on every visitor. CrackuEx follows that rule. Railway content is present because the overlap is real; it is not used as a licence to blur every notification into one leaderboard.[7]

Computer-based testing has also changed what "practice" must look like. A paper booklet allows circling and returning. A CBT adds a timer, a question palette and the knowledge that an unanswered item is simply a blank. Mock modules in a preparation app are useful only when they preserve those constraints and when the score is computed from counts of correct, wrong and blank answers—not from a number the browser volunteers.

1.3 Problem Statement

The problem addressed by this project can be stated as follows. Aspirants of SSC and related examinations need a single web workspace in which they can (a) attempt short drills whose answers are graded on the server, (b) read and write syllabus notes, (c) add their own multiple-choice questions to a personal drill bank, (d) sit topic tests and full-length mocks, and (e) see a conservative progress picture—Mastered, Reviewing or Needs work—derived from scores they actually earned. Existing sites often fail one or more of these tests: they leak answers to the client, they substitute predicted ranks for evidence, or they are assembled as a monolith that cannot be operated with separate web, worker and data processes.

A second, software-engineering problem sits beside the learning problem. Student projects frequently demonstrate screens but not failure behaviour. They hard-code secrets, log passwords, ignore SIGTERM, and treat Redis as a magic import. When such a system is later placed on a public host it fails in ways that cannot be debugged. CrackuEx therefore treats structured errors, redacted logs, health endpoints and a Compose file with healthchecks as part of the problem, not as optional decoration.

1.4 Need for the Study

Three observations justify the work. First, access to coaching centres is uneven. A free or low-friction web tool cannot replace a teacher, but it can give a remote candidate a daily drill and a place to keep notes. That is an equity argument and it is consistent with the spirit of SDG 4.[1,2] Second, the shift to CBT means that practice which does not look like a computer test is incomplete. Third, the MCA curriculum expects a full-stack artefact: analysis, design, implementation, results and a written report that another examiner can read without the author's verbal commentary. A tutorial clone of a blog does not meet that bar. A named product with a module map, tests and an operations story does.

There is also a pedagogical need inside the app. Mastery learning and the revised Bloom taxonomy both warn against treating every item as equal.[14,17] A candidate who scores well on one arithmetic topic and poorly on another should see that split, not a single vanity percentage. Keller's ARCS model further suggests that attention and confidence collapse when feedback is either absent or inflated.[15] CrackuEx uses three progress labels and refuses rank theatre for that reason.

1.5 Aim of the Project

The aim of the project is to design, implement and evaluate a web-based competitive examination preparation platform (CrackuEx) that supports authentic practice, notes, user-authored questions and evidence-based progress tracking for SSC-centred aspirants, with a production-minded Express API, MongoDB persistence, Redis-backed operational features, and a Docker Compose deployment path.

1.6 Objectives

1 To study the preparation needs of SSC and overlapping RRB aspirants and to translate those needs into a bounded product scope.

2 To review literature on e-learning, computer-based testing, adaptive practice, gamification and web application security in order to locate a research and design gap.

3 To specify functional and non-functional requirements and to produce an architecture that separates routes, validation, services and persistence.

4 To implement learner features: authentication, drills, syllabus notes, topic tests, mocks, analytics, MCQ battle, reminders and an admin question bank.

5 To implement engineering controls: JWT with tokenVersion, challenge-token grading, Helmet and CORS, DOMPurify, pino logs with redaction, Redis rate limits, a worker process, health checks and graceful shutdown.

6 To discuss results against the objectives, state limitations honestly, and propose a future scope that does not depend on unpublished miracles.

1.7 Scope of the Project

The present release is a complete learning loop for a signed-in user and a limited guest preview (catalogue of subjects and guest drills). It is not a payment gateway, not a social network, and not an official SSC portal. Table 1.2 records the boundary in one place so that later chapters are not accused of missing features that were never promised.

In scope	Out of scope in this submission
Email/password, OTP and Google sign-in; Bearer JWT	UPI / card payments and subscription plans
Daily drills with server-side grading	Official SSC answer keys or live exam delivery
Syllabus notes, topic tests, full mocks	Predicted All-India Rank or unofficial cut-off
User-added MCQs; admin question tools	Native iOS / Android stores (the UI is responsive web)
Progress labels and attempt history	Kubernetes, Kafka or multi-region active-active
Docker Compose with API, worker, Mongo, Redis	Guaranteed behaviour under Facebook-scale load

Table 1.1: Scope included in the present release and items left out of scope

1.8 Alignment with Sustainable Development Goals

Lovely Professional University requires project work to show a relationship with the United Nations Sustainable Development Goals where such a relationship is genuine. CrackuEx is not a campus solar plant or a clinical trial; the honest mapping is educational and, secondarily, occupational.

SDG	How the project relates	Limit of the claim
SDG 4 Quality Education	Free web practice, notes and topic feedback for government-exam aspirants, including guest drills before login.	The app does not replace a school or certify a degree.
SDG 8 Decent Work and Economic Growth	SSC and RRB examinations are documented public-employment routes; better practice can reduce wasted attempts.	The app does not recruit, shortlist or appoint any candidate.

SDG	How the project relates	Limit of the claim
SDG 9 Industry, Innovation and Infrastructure	A containerised API, worker and data plane is a small example of maintainable digital infrastructure.	This is a student-scale system, not national public digital infrastructure.

Table 1.2: Mapping of the project to selected Sustainable Development Goals

1.9 Nature and Type of the Project

As classified in the University guideline, this work is a **web-based** project with a **general / skill-oriented** character. It produces a running software system and a written report. It is not a pure survey, not a laboratory experiment on human subjects, and not an entrepreneurial company with audited accounts. If a later student wishes to take CrackuEx toward a start-up, that would be a new phase with legal, payment and content-licensing work that this submission does not pretend to have finished.

1.10 Organisation of the Report

Chapter 2 reviews literature on e-learning, CBT, mastery, gamification, REST-style web systems and security, and it states the gap that CrackuEx occupies. Chapter 3 presents analysis and design: requirements, architecture, data and deployment. Chapter 4 describes the implementation of each major module. Chapter 5 reports results and discusses them against the literature and the objectives. Chapter 6 concludes and records future scope. Publication details, annexures and IEEE references follow the last chapter as required by the guideline.

CHAPTER 2

REVIEW OF LITERATURE

2.1 Introduction to the Review

A project report that jumps from a slogan to a screenshot is not a review of literature. The purpose of this chapter is to locate CrackuEx among established ideas: how people learn from screens, how computer-based tests differ from paper, when "adaptive" practice is real and when it is a label, what gamification actually means, and which web-architecture and security sources a student implementation is entitled to cite. Only sources that exist as books, papers or dated official pages are used. No unpublished article is invented.[1]

2.2 E-Learning and Instructional Design

Clark and Mayer summarise a large body of multimedia evidence: people learn better when words and relevant graphics work together, when extraneous decoration is cut, and when practice is paired with explanation rather than with a scoreboard alone.[1] Ally's account of online-learning theory similarly insists that a platform is not "instructional" merely because it is online; activity, feedback and learner control have to be designed.[2] The United States Department of Education meta-analysis of online learning found advantages for blends of online and face-to-face study in many settings, with the usual warning that quality of design dwarfs the mere fact of being digital.[3]

Those findings caution a student project in two ways. First, a landing page full of animation is not evidence of learning. CrackuEx therefore keeps the public site informative and keeps the workspace practical: drills, notes, tests, progress. Second, feedback must be about the work the learner just did. An AI explanation endpoint in this project is offered only after a wrong drill answer, not as a chatbot that replaces the question bank. That is a modest reading of the e-learning literature, not a claim that the system is an intelligent tutor in the research sense.

2.3 Mastery, Taxonomy and Motivation

Bloom's taxonomy, and the later revision by Anderson and Krathwohl, remain the common language for talking about what a question is asking a learner to do.[14,17] Most SSC items in a first-level CBT sit at remember, understand and apply. A preparation app that only stores items can still respect the taxonomy by grouping practice under syllabus topics and by not

mixing a vocabulary drill with a full mock as if they were the same cognitive event. Mastery learning, associated with Bloom's later writing, argues that time should vary while the standard stays fixed. The three-state progress model in CrackuEx (Needs work, Reviewing, Mastered) is a software-sized echo of that idea: a topic is not "done" because it was opened; it is classified from a topic-test score.

Keller's ARCS model (Attention, Relevance, Confidence, Satisfaction) is often cited in instructional-design courses.[15] Confidence is the element most abused by coaching advertisements. If a product manufactures confidence with fake percentiles, it violates the model it claims to follow. CrackuEx's product copy on the landing page is therefore written to describe what the software does, not what rank the Commission might one day publish.

Siemens' connectivism is sometimes invoked to justify every social feature in an educational app.[16] The present project cites it only to note the opposite design choice. CrackuEx is not a learning network of posts and followers. The one social-adjacent module is a timed MCQ battle with a leaderboard of scores the server computed. That is competition, not connectivist pedagogy, and the report does not pretend otherwise.

2.4 Computer-Based Testing and Practice Design

A computer-based test is not a PDF displayed in a browser. It is a timed, item-based session whose scoring rules are part of the examination notice. Official SSC and RRB pages are the proper source for those rules.[6,7] Academic work on CBT emphasises item security, consistent timing and the separation of authoring from delivery. For a preparation clone the ethical bar is lower than for a live public examination, but the engineering bar is similar in one respect: the client must not be trusted to know the key while the item is on screen.

That last sentence drove a concrete design in Chapter 4. Drill routes return a challengeToken and no answer. Competition routes return a sessionToken and no keys. Verification happens on the server. The literature on adaptive hypermedia, notably Brusilovsky, describes systems that change the next item from a model of the learner.[4] CrackuEx is only weakly adaptive: it can prefer weak topics and it keeps a wrong-answer log. It does not estimate a latent ability on an item-response curve. Calling it "AI adaptive testing" would be a literature violation, so this report does not.

2.5 Gamification

Deterding and colleagues defined gamification as the use of game design elements in non-game contexts, and they warned against reducing the idea to points and badges.[5] In examination preparation, a leaderboard can increase time-on-task for some learners and

anxiety for others. CrackuEx therefore confines game-like structure to an optional MCQ battle and to short drills. Progress in the syllabus is not a badge; it is a label tied to a score. That is a conservative reading of the gamification paper, not an attempt to publish a new theory of play.

2.6 Web Architecture for Student-Scale Products

Fielding's dissertation remains the reference for REST as an architectural style for network-based software.[8] CrackuEx's API is resource-oriented (auth, drill, study, mock, and so on) and JSON over HTTP. It is not a pure REST textbook: some actions are verbs because grading a challenge is an action. Express is the server framework; React is the client library; MongoDB and Redis are the data and cache stores. Each of those tools publishes documentation that an implementation report may cite as an engineering source, not as a research article.[10,11,12,13,19,20]

Single-page applications move navigation into the client. That choice reduces full-page reloads and complicates security: tokens live near JavaScript, and HTML injected into notes becomes a cross-site scripting risk. The implementation chapter records the mitigations that were actually built (DOMPurify, helmet headers, CORS allow-lists, password-reset tokens in the URL hash). Docker documentation supplies the vocabulary for multi-container development, healthchecks and non-root images.[18] None of this literature claims that a student Compose file is equivalent to a hyperscale platform. It claims only that the operators of small systems should still use those patterns.

2.7 Security Literature Relevant to the Project

The OWASP cheat-sheet series is the practical security literature most used by working web teams. The JSON Web Token notes, in particular, warn against treating a JWT as a complete session system and against storing secrets in source control.[9] CrackuEx uses a Bearer access token with a typ claim and a tokenVersion stored with the user. A new login bumps the version so that an old token fails even if it has not yet expired. Passwords are hashed; OTP and reset flows are logged without the secret values because the logger redacts those field names.

Rate limiting is another OWASP-adjacent concern. An API without a counter is an invitation to credential stuffing. The project uses Redis INCR with a TTL for hosted deployments and a memory fallback only for local development, with a warning when Redis is required but not ready. That is not a novel algorithm; it is a decision to follow common advice rather than to leave the door open.

2.8 Existing Practice and the Research Gap

Commercial Indian examination portals vary widely. Some offer large item banks and paid analytics. Some scrape Telegram channels. A student project cannot fairly benchmark against a private company's unpublished conversion rates, and this report does not invent such numbers. What can be said from public observation is that many sites (a) mix unofficial cut-offs with practice, (b) grade in the browser, or (c) provide neither notes nor a way for the learner to add a question they met in class.

The gap that CrackuEx tries to fill is therefore not "the first website for SSC." That claim would be false. The gap is a **transparent, small, inspectable system** in which practice, notes, user-authored items and three-state progress live together, answers are not shipped to the client during an attempt, and the operations story (Compose, Redis, worker, health, shutdown) is part of the academic deliverable. Literature on e-learning supplies the pedagogical caution; literature on REST and OWASP supplies the engineering caution; official SSC/RRB pages supply the domain facts.[1,6,8,9]

2.9 Summary of the Review

The reviewed sources agree on a few points that guided design. Online practice helps when feedback is about the task. Mastery should be earned. Gamification is optional and easy to misuse. CBT practice must not leak keys. Web APIs should be explicit about resources and errors. Tokens are not a substitute for server-side checks. CrackuEx is an application of those points, not a new theory of learning.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter converts the problem and the literature gap into a system that can be built. It records feasibility, users, functional and non-functional requirements, the logical architecture, the main collections, and the deployment topology. Implementation details that belong to code are reserved for Chapter 4; the aim here is to show that those details follow a design rather than an accident.

3.2 Feasibility

Technical feasibility is high because the chosen tools are mature: a JavaScript runtime, a document database, a cache, and a well-documented front-end compiler.[11,13,19,20] Operational feasibility for a student host is acceptable: Compose brings four processes up on one machine; a later split onto a platform-as-a-service is possible because the API is stateless aside from JWT and Redis. Economic feasibility is acceptable for an academic project: there is no payment module and no licence fee for the core stack. Legal feasibility requires care. The application must not present itself as an SSC property, must not scrape copyrighted test booklets into the public bank without rights, and must allow users to add their own questions under their account. Official pattern facts are taken from public notices, not from pirated PDFs.[6,7]

Schedule feasibility fitted an MCA project calendar: a vertical slice (auth, one drill, one notes path) first, then mocks, competition, reminders, hardening and the written report. The hardening pass—healthchecks, worker split, redacted logs, DOMPurify, tests for unauthenticated calls—was treated as in-scope because a web project that cannot be restarted cleanly is not complete.

3.3 Users and Roles

Two roles exist. A **user** is an aspirant. After authentication the user practises drills, reads and writes notes, sits topic tests and mocks, inspects performance, joins an MCQ battle, and sets reminders. A **guest** may browse subjects, topics and some notes, and may try guest drills that are signed as a guest identity so that the same grading path is exercised without a full account. An **admin** is identified by a configured email address and may manage

exam–subject mapping, question upload and progress export. There is no third “institute owner” role in this release; adding one would be a product decision, not a weekend patch.

3.4 Functional Requirements

ID	Requirement
FR-01	Register and log in with email/password; optional OTP; optional Google sign-in.
FR-02	Issue a Bearer access token; invalidate prior sessions via tokenVersion.
FR-03	Reset password using a token carried in the URL hash, not a query string.
FR-04	Serve the next drill item with a challenge token and no answer key.
FR-05	Verify a drill answer on the server and record wrong items.
FR-06	Allow a signed-in user to add an MCQ to the personal drill bank.
FR-07	Browse subjects, topics and notes; sanitise note markup on the client.
FR-08	Deliver a topic test and classify the topic as Mastered / Reviewing / Needs work.
FR-09	Deliver full-length mocks; persist counts, not a client-invented score.
FR-10	Show attempt history and simple analytics charts.
FR-11	Run a timed MCQ battle; grade with a session token; show a leaderboard.
FR-12	Create study reminders; process due items in a worker when Redis is available.
FR-13	Explain a wrong drill answer through a dedicated AI route when configured.
FR-14	Allow an admin to manage questions and export progress CSV.
FR-15	Expose /health and /api/health with database and Redis status.

Table 3.1: Functional requirements of CrackuEx

3.5 Non-Functional Requirements

ID	Concern	Design response
NFR-01	Security	Helmet, CORS allow-list, Zod validation, mongo sanitise, HPP, JWT secret from environment.
NFR-02	Integrity of scores	Challenge/session tokens; server grading; progress from counts.
NFR-03	Observability	pino JSON logs, requestId on every error body, redaction of secrets.
NFR-04	Operability	Docker healthchecks, non-root image, SIGTERM shutdown, worker split.

ID	Concern	Design response
NFR-05	Resilience (local)	In-process reminder cron and memory rate-limit only when not hosted.
NFR-06	Usability	English-first UI; three progress labels; toasts with error codes.
NFR-07	Maintainability	Feature folders; controllers do not pass req into services.
NFR-08	Performance (honest)	Compression, Redis counters, indexes via Mongoose; no fake “million user” claim.

Table 3.2: Non-functional requirements and the mechanism used to address each

3.6 Logical Architecture

The system is a browser client, a JSON API, a document database, a Redis instance, and an optional worker. Figure 3.1 shows the layers. The client never talks to MongoDB. The worker never serves HTTP. The API is the only process that validates user input with Zod and that writes progress.

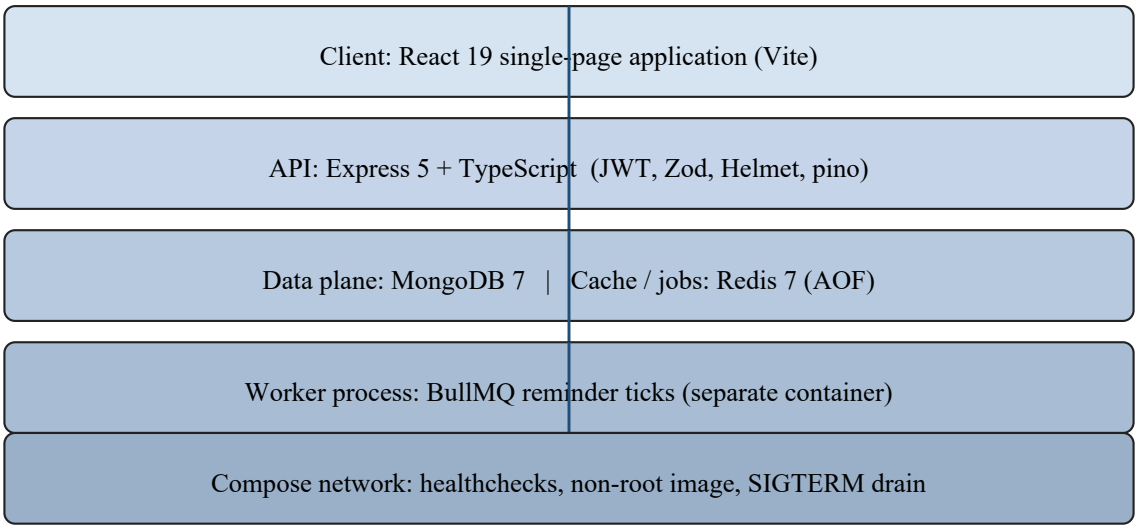


Figure 3.1: Logical architecture of CrackuEx

Inside the API a request meets middleware first (request identifier, Helmet, compression, CORS, JSON body limit, sanitise, HPP, pino-http, optional rate limit). It then hits a route file. Mutating routes declare a Zod schema. The controller maps the authenticated user and the body into a DTO. The service applies rules and throws a typed HttpError on failure. The repository is the only layer that imports Mongoose models. That pipeline is Figure 3.2. It exists so that a defect can be placed: a bad status code is a controller or error-middleware problem; a wrong grade is a service problem; a wrong query

is a repository problem.

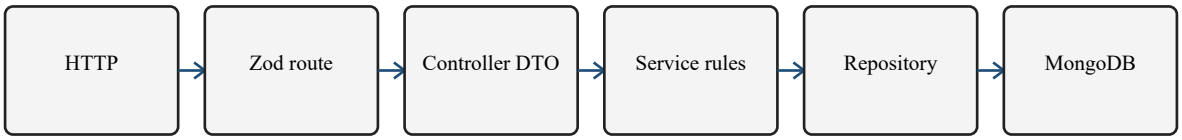


Figure 3.2: Request path from the browser to persistence

3.7 Client Architecture

The front end is a Vite-built React 19 application. Transport lives in one module (apiService): it attaches the Authorization header, applies a timeout, and clears the local session on HTTP 401. Feature API helpers (auth, drill, competition, study) sit above transport. Feature hooks call those helpers. Screens are lazy-loaded inside the dashboard shell and wrapped in an error boundary so that a crash in analytics does not destroy notes. Navigation in the signed-in workspace is view-state inside the shell rather than a deep public URL tree. That is a deliberate SPA choice and it is documented so that a maintainer does not search for React Router paths that do not exist for every pane.[11,19]

3.8 Data Design

MongoDB stores documents, not a third-normal-form warehouse. The important design rule is identity: progress, competition scores and reminders key off **userId**, not off a display name that the user might later change. Usernames may be copied onto a leaderboard row for display. Table 3.3 lists the principal collections as they are used in code.

Collection (logical)	Role
users	Credentials, tokenVersion, role, profile fields, optional avatar.
subjects / topics	Syllabus tree for an exam; notes hang off a topic.
questions / TCS items	MCQ bank used by drills, topic tests and admin tools.
drill performance / wrong log	Per-user drill history and items to retry.
mock tests / mock progress	Paper definition and attempt records with counts.
syllabus progress	Topic-test scores and Mastered/Reviewing/Needs work.
competition sessions	Battle papers and server-graded results.
reminders / notifications	User-scheduled study alarms.

Collection (logical)	Role
otp / password-reset	Short-lived secrets; never logged in plaintext.

Table 3.3: Principal MongoDB collections and their role

Redis is not a second database of record. In this design it holds rate-limit counters with a TTL, optional cache entries, and BullMQ job state when reminders run out-of-process. Redis AOF is enabled in Compose so that a container restart does not always empty the queue. Hosted deployments require REDIS_URL; a silent memory map in production would hide an outage, so that path is refused.[12]

3.9 Progress Model

After a topic test the application stores a score and a status. The learner-facing labels are Needs work, Reviewing and Mastered. They correspond to internal colour codes (red, yellow, green) but the UI speaks in words because a colour-only code fails accessibility and fails a printed report. The labels are not ranks. They are not cut-offs. They are a coarse bucket over a score the server accepted.

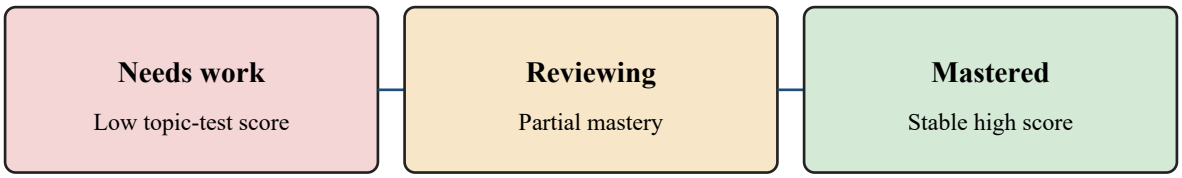


Figure 3.3: Progress states used after a topic test

3.10 Security Design

Security is treated as a set of checks that fail closed. CORS allows a configured front-end origin (and localhost variants when not hosted). JSON bodies are capped at two megabytes. Helmet sets baseline headers; a Permissions-Policy header disables camera and microphone. Passwords never return from the API. JWT signing uses an environment secret; Compose refuses to start the API if JWT_SECRET is missing. Reset links put the token in the hash fragment so that intermediate access logs are less likely to keep it. Notes are sanitised in the browser before inner HTML is used. These choices follow common OWASP advice; they are not a formal penetration test.[9]

3.11 Deployment Design

Local production-like bring-up uses Docker Compose. MongoDB 7 and Redis 7 start with healthchecks. The API image is multi-stage, runs as a non-root user, and declares a HEALTHCHECK against /health. A second service uses the same image with a worker command so that reminder ticks do not share the HTTP event loop. REMINDER_INLINE=0 on the API makes that split real. Chapter 4 returns to the Compose file as an implemented artefact rather than a wish.

CHAPTER 4

IMPLEMENTATION OF THE PROJECT

4.1 Introduction

Implementation is the chapter in which design either becomes code or becomes a story. This chapter follows the running system: the stack, the module convention, authentication, each learner feature, the admin path, Redis and Docker, errors and logs, and tests. Screen-by-screen marketing language is avoided. Where a feature is incomplete, the text says so rather than filling the page with a mock-up.

4.2 Technology Stack

Layer	Choice	Reason in this project
UI	React 19 + TypeScript + Vite	Component model, typed props, fast local builds.[[11,19]]
Charts / motion	Recharts, GSAP (landing)	Progress charts; landing motion only.
Markup safety	DOMPurify	Notes may contain HTML; sanitise before display.
API	Express 5 + TypeScript	Small, explicit HTTP surface.[[13]]
Validation	Zod	Schemas next to routes; no informal req.body casts.
Database	MongoDB 7 + Mongoose	Document shape fits questions and notes.[[10]]
Cache / jobs	Redis 7 + ioredis + BullMQ	TTL counters and reminder queue.[[12]]
Auth	JWT + bcrypt + optional Google	Bearer access; password hash; federated option.[[9]]
Logs	pino / pino-http	JSON logs with redaction; pretty print only locally.
HTTP hardening	helmet, hpp, cors, compression	Headers, parameter pollution, origin checks.
Runtime pack	Docker Compose	Mongo, Redis, API, worker with healthchecks.[[18]]
Tests	Vitest	Unit tests for cache, rate-limit store and auth error shape.

Table 4.1: Technology stack used in the implementation

4.3 Backend Module Convention

Every feature folder under the API source tree is meant to be read in one sitting: routes, Zod schema, controller, service, repository, model, interfaces. Shared tools live in a library folder (logger, validate, HttpError, challenge tokens, request identifiers). Controllers are forbidden from passing the raw Express request into a service. Services are forbidden from writing HTTP status codes; they throw. The error middleware turns those throws into a JSON body of the form status, code, message, requestId. A one-year engineer can follow a bug from the status code to the throw site without hunting through res.status calls scattered in helpers.

Public routes include health, Google client configuration, authentication, and a read-only catalogue (subjects, topics, notes, mock list, competition leaderboard, guest drills). Protected mounts wrap the feature router with a database-ready check and requireAuth. Guest drills reuse the same verify schema as signed-in drills so that grading logic is not copied. Table 4.2 summarises the groups.

Mount	Auth	Purpose
/api/health	Public	Process liveness plus Mongo and Redis flags.
/api/auth	Mixed	Register, login, me, OTP, password, Google, progress, admin.
/api/study	Optional then protected	Catalogue, notes, user study writes.
/api/drill	Guest or user	Next item, verify, personal questions.
/api/mock	Public read; protected attempt	Papers and scored attempts.
/api/competition	Mixed	Battle paper, submit, leaderboard.
/api/ai	Protected	Explain a wrong answer when a provider is configured.
/api/reminders	Protected	CRUD for study alarms.
/api/prep, /exam-config, /questions	As routed	Prep helpers, exam config, TCS-style items.

Table 4.2: Representative API route groups

4.4 Authentication and Session Control

Registration stores a bcrypt hash. Login verifies the hash, increments tokenVersion, and returns a JWT with typ=access. Subsequent requests send Authorization: Bearer. The auth middleware rejects a missing header with the same error envelope as any other failure

(UNAUTHORIZED plus requestId), which is covered by a test. Google sign-in is optional and depends on a client identifier (and a client secret only if the code flow is enabled). The front end asks /api/auth/google-config so that a missing Google setup disables the button instead of throwing in the browser.

Password reset constructs a front-end URL that carries the token in the hash (#reset=…). Hash fragments are not sent to the server on the next static-file request, which is why they are preferred to a query string for this use. OTP mail uses SMTP when configured; if SMTP is absent the API logs a warning rather than pretending that mail was sent. Avatars are loaded only when /auth/me?include=avatar is requested, so the common session check stays small.

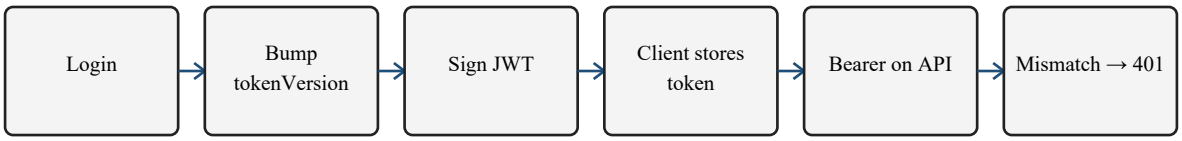


Figure 4.1: Authentication and session control

The client still keeps the JWT in localStorage. That is a known trade-off: it is simple and it is visible to any script that achieves script injection. The mitigations are sanitised notes, a tight CORS list, and CSP on hosted deployments. An httpOnly cookie session would be a reasonable future change and is listed in Chapter 6 rather than described as already done.

4.5 Drill Engine

Daily drills are the smallest honest practice loop. GET /drill/next (or the guest equivalent) selects an item and signs a challenge that binds the item to the user identity. The JSON returned to the browser has the stem, the options and the challengeToken. It does not have the key. POST /drill/verify accepts the token and the user's option, checks the signature, grades, and updates drill performance and the wrong log. A user may add an MCQ of their own; that item joins the bank used for later drills rather than living only in a forgotten notepad.

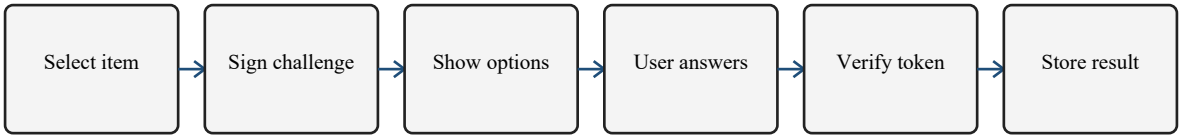


Figure 4.2: Drill challenge and server-side grading

The same pattern appears in competition: questions ship with a sessionToken; submit sends answers; the server grades. Copying this pattern twice is intentional. A single "trust the client score" helper would have been shorter and wrong. Integrity of practice is a functional requirement (FR-04, FR-05, FR-11), not a style remark.

4.6 Syllabus, Notes and Topic Tests

Study data is a tree: exam, subjects, topics, notes. Guests may read. Signed-in users may write quick notes and keep roadmap notes attached to a topic. Because notes are rendered as rich text, the client runs DOMPurify before display. That does not make the API a full HTML sanitiser of record; it does close the most likely XSS window in this application, which is a note that contains a script tag.

A topic test is a short, syllabus-bound paper. On completion the service stores score, maximum, elapsed time and attempt number, and it assigns the three-state label. The performance workspace later aggregates how many topics sit in each bucket. The landing page describes this loop in the same words the software uses, so that a visitor is not promised a rank engine that the dashboard does not contain.

4.7 Mocks, Performance and Analytics

Full mocks are longer papers with a timer. Persistence records correct, wrong and blank counts. Accuracy is derived. A free-form score field supplied by the client is not the source of truth. CSV export for administrators prefixes formula-looking cells so that a spreadsheet does not execute a pasted payload—a small but serious hygiene step when progress leaves the database.

Analytics charts use attempt history. They are descriptive. They do not run a psychometric model. If a future version adds item-response theory, it will need a new chapter of validation; this version would be dishonest if it used that vocabulary today.

4.8 MCQ Battle

The competition module is a timed quiz. The leaderboard is public-readable so that a visitor can see that the feature exists; submitting a paper requires a signed-in user. Display names may appear on the board; identity for the write path remains userId. This is the only place the product is allowed to look like a game, in line with the caution in Chapter 2.[5]

4.9 Reminders and the Worker

Reminders are user-owned documents with a time. When Redis is configured and ready, BullMQ runs a repeatable tick in a worker process. When Redis is absent in local development, an in-process cron performs the same tick. Hosted environments require Redis and do not quietly fall back, because a silent fallback would look healthy while dropping mail. Compose sets REMINDER_INLINE=0 on the API and runs dist/worker.js in a second container. Mail, if SMTP is set, is sent through a small reminder-mail helper that uses the same redacting logger as the rest of the API.

4.10 AI Explanation Path

POST /api/ai/explain is a protected route. It is intended to comment on a wrong drill item, not to generate an entire question paper. If the upstream key is missing, the service fails with an application error rather than returning an invented explanation. Logs of this path go through pino. The feature is optional at runtime and is not the academic centre of the project. The centre remains graded practice and progress.

4.11 Administration

An administrator, recognised by ADMIN_EMAIL, can map exam subjects, upload questions and export progress. Ordinary users never see those routes as working tools. Role checks live on the server; hiding a sidebar icon is not access control. This is an ordinary web rule and it is easy to forget in SPAs, which is why it is written down here.

4.12 Docker, Redis Lifecycle and Shutdown

The backend Dockerfile is multi-stage. The final image does not include development dependencies, does not run as root, and defines a HEALTHCHECK. A .dockerignore file keeps tests, local env files and documentation out of the build context. Compose wires four services. Mongo and Redis must become healthy before the API and worker start. Redis runs with appendonly yes and a named volume. JWT_SECRET is required from the environment; it is not committed.

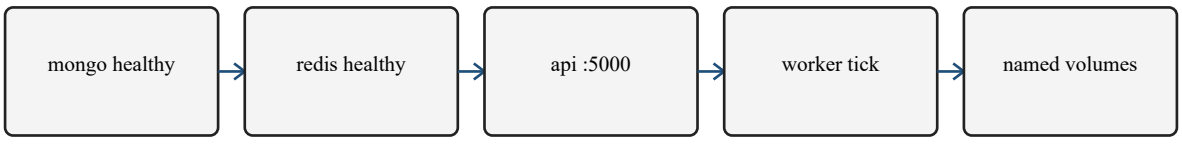


Figure 4.3: Docker Compose runtime (API, worker, MongoDB, Redis)

The Redis client reconnects, exposes waitForRedis and closeRedis, and reports readiness. Rate limiting uses a Lua INCR plus PEXPIRE so that a counter cannot live forever without a TTL. Cache helpers may use memory locally; they warn when Redis fails. Process shutdown on SIGTERM and SIGINT stops accepting work, closes Redis and disconnects Mongo. That is the difference between a demo script and a process a container orchestrator can replace.

4.13 Errors, Logging and Front-End Hardening

Every error body carries status="error", a stable code, a message and requestId. Auth and database middleware call next(HttpError) rather than sending a one-off shape. pino redacts authorization headers, cookies, password, token, otp and REDIS_URL. Pretty-printing is skipped in test and in hosted runtime so that log shippers receive JSON. The front end maps those errors to toasts that include the code and a short request reference, which makes a screenshot useful to a developer.

Control	Where it lives
Helmet + Permissions-Policy	API application setup
CORS allow-list / blocked origin log	API application setup
Zod on mutating routes	Feature schemas
mongo sanitise + HPP	API middleware
JWT + tokenVersion	Auth module
Challenge / session tokens	Drill and competition
DOMPurify on notes	Front-end notes rendering
Reset token in URL hash	Password service + Auth panel
Rate limit store on Redis	Hosted /api limiter
Logger redaction	pino configuration

Table 4.3: Security controls implemented in the current codebase

4.14 Testing during Implementation

Automated tests use Vitest on the API. They cover the memory path of the cache, the rate-limit store, and the JSON shape of an unauthenticated call (UNAUTHORIZED and requestId). They do not replace a live Redis integration suite, and this report does not

inflate fourteen passing tests into a quality certificate. They do lock the error contract and the fallback behaviour that production hardening depends on. Typecheck scripts on both packages are part of the daily build, not an afterthought.

CHAPTER 5

RESULTS AND DISCUSSIONS

5.1 Introduction

Results for a software project are not a laboratory table of yields. They are evidence that the requirements of Chapter 3 exist as running behaviour, plus a discussion of what that behaviour means against the literature and against the limits of a student deployment. This chapter therefore lists completed modules, shows how health and tests were read, and argues—without advertising language—where the work succeeded and where it did not.

5.2 Completed Learner Loop

The intended loop was: pick an exam, practise, add a question if needed, write a note, sit a topic test, sit a mock, inspect progress. Figure 5.1 restates that loop. Table 5.1 maps each step to an implemented surface. The landing page uses the same vocabulary as the workspace so that a visitor is not sold a different product from the one they will log into.



Figure 5.1: Learner study loop supported by the application

Module	Evidence of completion
Authentication	Register/login/OTP/Google config routes; JWT with tokenVersion; reset hash.
Drills	challengeToken issue and verify; guest path; user-authored items.
Notes / syllabus	Subject–topic–notes tree; DOMPurify on render; topic tests.
Mocks	Paper fetch and attempt records with counts.
Performance / analytics	History workspace; Mastered / Reviewing / Needs work counts.
MCQ battle	Session-token grading; leaderboard.
Reminders	CRUD plus BullMQ worker or local cron.
Admin	Protected question and export tools gated by admin email.
Operations	Compose, health, shutdown, redacted logs, rate-limit Redis store.

Table 5.1: Implemented learner-facing modules and the evidence of completion

5.3 Operational Results

The health endpoints return a compact JSON object. An operator can distinguish “process is up” from “database is connected” from “Redis is still connecting.” That distinction matters. A reverse proxy that only checks TCP would restart nothing while Mongo is down; a probe that requires db=connected will fail the container until the dependency is actually usable. Table 5.2 is the reading guide.

Field	Healthy reading	Unhealthy reading
status	ok	degraded (HTTP 503 when Mongo is down)
db	connected	disconnected
redis	connected, or disabled if no REDIS_URL locally	connecting while REDIS_URL is set
uptime	increases while the process lives	resets after a crash loop

Table 5.2: Health endpoint fields and how an operator should read them

Compose start with a generated JWT secret brings Mongo, Redis, API and worker up as four containers. The API does not embed reminder ticks when REMINDER_INLINE=0. This is a qualitative result: the process boundaries in Chapter 3 exist at runtime, not only in a diagram.

5.4 Test Results

Suite	What passed	What it does not prove
API Vitest (14 tests in the hardening pass)	Cache memory path; rate-limit store; unauthenticated call returns UNAUTHORIZED with requestId.	Full end-to-end grading of every exam paper; live Redis failover; load test.
TypeScript build	frontend tsc and backend tsc clean at the time of this report’s engineering snapshot.	Runtime absence of logic errors.

Table 5.3: Automated tests executed during this project

Fourteen tests are a beginning. They were written to pin behaviour that is easy to break during a “quick fix”: error JSON shape and the local fallback of Redis helpers. Expanding coverage to drill verify, competition submit and reminder enqueue is listed as

future work because claiming that coverage now would be false.

5.5 Discussion against Objectives

Objective 1 asked for a bounded SSC-first product. The landing and the workspace both speak in official exam names and refuse rank theatre. That objective is met at the level of product language and module set. Objective 2 asked for a literature review that locates a gap. Chapter 2 meets it without inventing papers. Objective 3 asked for requirements and a layered API. Chapters 3 and 4 show that layout in folders that actually exist. Objective 4 asked for the learner features; Table 5.1 is the result. Objective 5 asked for engineering controls; Table 4.3 and the Compose file are the result. Objective 6 is this chapter and the next.

Where the work is weaker is scale evidence and content volume. The architecture can sit behind more than one API replica because sessions are JWTs and counters are in Redis, but no k6 or JMeter study is attached. Question-bank size depends on what an admin or a user loads; the platform is not a published SSC archive. Those are not secret failures. They are the difference between a maintainable student system and a commercial item bank.

5.6 Discussion against Literature

Clark and Mayer would still find the landing page more decorative than a worksheet.[1] The defence is that decoration is confined to the public site; the workspace is drills and notes. Brusilovsky would not call the drill picker an adaptive hypermedia system.[4] The defence is that the report never used that name. Deterding would recognise the battle module as a thin gamification layer and would ask whether it helps learning or only time-on-site.[5] The honest answer is that this project did not run a randomised trial, so it does not know. OWASP would still dislike localStorage tokens.[9] Chapter 6 records a cookie session as future work rather than arguing that the current choice is ideal.

On official sources, the project is aligned with the literature of record: examination facts come from SSC and RRB public pages, dated in the reference list.[6,7] That is a small scholarly habit with a large practical effect. It stops the report from repeating social-media cut-offs as if they were Commission policy.

5.7 Limitations

Several limitations are built in. The application is a single-page workspace; deep public URLs for every pane are limited. Content quality equals whatever questions are loaded.

SMTP is optional; without it, OTP and reminder mail do not leave the machine. Google sign-in is optional. The AI explainer is optional and only as good as the configured model. There is no payment plan, which is a scope choice, not a missing bugfix. There is no formal accessibility audit, though progress is labelled in words as well as colour. There is no claim of compliance with a government security audit. Browser verification of every visual change was part of development practice; this PDF cannot embed a live session.

A further limitation is evaluation with learners. No classroom experiment was run. No ethics-board study of minors was attempted, nor is one appropriate for this codebase. Claims of “better marks” would therefore be malpractice. The result that can be claimed is that a candidate can practise, note, add an item, and see a three-state topic map produced from scores.

5.8 Summary of the Discussion

CrackuEx meets its stated requirements as a web-based MCA project. It is stricter than many student portals about scoring integrity and operations, and it is more modest than commercial portals about ranks. The right way to read the results is as a working system with tests and a deployment file, not as a finished national platform.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project set out to build a web-based preparation workspace for SSC-centred aspirants and to write it down as an MCA dissertation-style report. The software, named CrackuEx, offers drills that are graded on the server, syllabus notes, user-authored questions, topic tests, mocks, a conservative progress model, an optional battle, reminders, and an admin path. The API is typed, validated, logged and containerised. Redis and a worker exist as real processes. Health and shutdown exist as real code. Those sentences can be checked against the repository; they are the conclusion that matters.

The academic conclusion is narrower. E-learning literature warned against decoration without practice and against fake confidence. Security literature warned against client-side keys and against secrets in images. Official examination pages warned against invented cut-offs. The implementation took those warnings as requirements. Where the project refused a fashionable claim—nationwide scale, full adaptivity, guaranteed score improvement—it did so on purpose. A degree report that cannot say “no” to a claim is not scholarship.

For the University’s classification, the work is a web-based, skill-oriented software project aligned primarily with SDG 4 and secondarily with SDG 8 and SDG 9, with the limits stated in Table 1.1. It is original in its composition as a system and as a text. It is not original as a new learning theory, and it does not need to be.

6.2 Future Scope

Future work should proceed in layers that the current architecture can actually accept. The first layer is evaluation: a small, consented study of adult learners measuring time-on-task and topic-label movement, not a fantasy of All-India Rank. The second layer is session hygiene: moving the access token to an httpOnly cookie with CSRF protection, which would address the localStorage residual risk. The third layer is testing: contract tests for drill verify, competition submit, and reminder enqueue against a test Redis.

The fourth layer is content operations: editorial workflow, duplicate detection beyond the present helpers, and a clearer licence line for every imported item. The fifth layer is delivery: a reverse proxy, managed MongoDB and Redis, and horizontal API replicas

behind the existing JWT and Redis counters. Only after those steps would a load test be meaningful. Naming Kubernetes or a message bus before those steps would be costume, not engineering.

Product ideas that are attractive but out of the present ethics line should stay out until they can be done correctly: live classes, paid tiers, and any display of a predicted cut-off. If payment is added, it must be a dedicated module with a provider, invoices and refunds—not a button that pretends to charge. If live classes are added, they are a different product with moderation and recording consent. CrackuEx can grow; it should not grow by lying about what it already is.

6.3 Closing Remark

A competitive examination will always be larger than an app. What an app can do is respect the candidate’s time: one more graded item, one note that is still there tomorrow, one topic marked Needs work without insult. CrackuEx was built to do that much, and to be operable while it does it. That is the whole of the claim.

PUBLICATION DETAILS

Nil. No research paper arising from this project has been published or accepted in a journal or conference at the time of submission. The student does not list preprints or informal blog posts as publications. If a paper is later produced from this work, it will be filed as a separate academic output and will cite this project report as the source of the system description.

ANNEXURES

Annexure A — Secrets and Environment

The following names must be supplied by the environment or by a root .env file that is not committed. The Compose file already refuses to start without JWT_SECRET.

Name	Purpose
JWT_SECRET	Signs access tokens. Generate a long random value per environment.
MONGODB_URI	Connection string. In Compose this points at the mongo service.
REDIS_URL	Required when hosted; used for limits, cache and BullMQ.
FRONTEND_URL / FRONTEND_URLS	CORS allow-list for the browser origin.
ADMIN_EMAIL	Account that receives the admin role after login.
GOOGLE_CLIENT_ID / SECRET	Optional Google sign-in.
SMTP settings	Optional mail for OTP and reminders.
AI provider key	Optional; only if /api/ai/explain is to be used.

Table A.1: Environment variables that must not be committed to source control

Annexure B — Standard Error Envelope

A typical unauthenticated response, as asserted by tests, has the following shape. The message text may vary; the code and the presence of requestId must not.

{ "status": "error", "code": "UNAUTHORIZED", "message": "Authentication required.", "requestId": "…" }

Annexure C — Suggested Local Bring-up

From the repository root, set a secret and start the stack. The command below is an example of operator practice; it is not an exploit and it does not bypass any system. Replace the secret generation with an equivalent on Windows if openssl is absent.

JWT_SECRET=<long random value> docker compose up --build

The API listens on port 5000. The Vite development client proxies /api to that port. Production front-end hosting is a static deployment of the compiled client with FRONTEND_URL pointed at that origin.

Annexure D — Cover and Declaration Note

Annexure-I of the University guideline is the cover page. It has been prepared as the first page of this PDF with the prescribed elements: topic, statement of partial fulfilment, name of the degree, student name, registration number 22407625451, the Centre for Distance and Online Education logo, Lovely Professional University, Phagwara, Punjab, and year of submission. Annexure-II is the declaration, bound immediately after the cover.

REFERENCES

[1] R. C. Clark and R. E. Mayer, e-Learning and the Science of Instruction: Proven Guidelines for Consumers and Designers of Multimedia Learning, 4th ed. Hoboken, NJ, USA: Wiley, 2016.

[2] M. Ally, "Foundations of educational theory for online learning," in The Theory and Practice of Online Learning, T. Anderson, Ed. Edmonton, AB, Canada: AU Press, 2008, pp. 15–44.

[3] B. Means, Y. Toyama, R. Murphy, M. Bakia, and K. Jones, Evaluation of Evidence-Based Practices in Online Learning: A Meta-Analysis and Review of Online Learning Studies. Washington, DC, USA: U.S. Department of Education, 2010.

[4] P. Brusilovsky, "Adaptive hypermedia," User Modeling and User-Adapted Interaction, vol. 11, no. 1–2, pp. 87–110, 2001.

[5] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness: Defining 'gamification'," in Proc. 15th Int. Academic MindTrek Conf., Tampere, Finland, 2011, pp. 9–15.

[6] Staff Selection Commission, "Official website," Government of India. [Online]. Available: https://ssc.nic.in/. Dated: 14/10/2026.

[7] Railway Recruitment Boards, "RRB official portal." [Online]. Available: https://www.rrbcdg.gov.in/. Dated: 14/10/2026.

[8] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. of California, Irvine, CA, USA, 2000.

[9] OWASP Foundation, "JSON Web Token Cheat Sheet for Java," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html. Dated: 14/10/2026.

[10] MongoDB, Inc., "MongoDB Manual." [Online]. Available: https://www.mongodb.com/docs/manual/. Dated: 14/10/2026.

[11] Meta Platforms, Inc., "React documentation." [Online]. Available: https://react.dev/. Dated: 14/10/2026.

[12] Redis Ltd., "Redis documentation." [Online]. Available: https://redis.io/docs/. Dated: 14/10/2026.

[13] OpenJS Foundation, “Express — Node.js web application framework.” [Online]. Available: https://expressjs.com/. Dated: 14/10/2026.

[14] B. S. Bloom, Ed., Taxonomy of Educational Objectives: The Classification of Educational Goals. Handbook I: Cognitive Domain. New York, NY, USA: Longmans, Green, 1956.

[15] J. M. Keller, “Development and use of the ARCS model of instructional design,” Journal of Instructional Development, vol. 10, no. 3, pp. 2–10, 1987.

[16] G. Siemens, “Connectivism: A learning theory for the digital age,” Int. J. Instructional Technol. Distance Learning, vol. 2, no. 1, 2005.

[17] L. W. Anderson and D. R. Krathwohl, Eds., A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom’s Taxonomy of Educational Objectives. New York, NY, USA: Longman, 2001.

[18] Docker, Inc., “Docker documentation.” [Online]. Available: https://docs.docker.com/. Dated: 14/10/2026.

[19] Vite, “Vite documentation.” [Online]. Available: https://vite.dev/. Dated: 14/10/2026.

[20] OpenJS Foundation, “Node.js documentation.” [Online]. Available: https://nodejs.org/docs/. Dated: 14/10/2026.

[21] United Nations, “Transforming our world: the 2030 Agenda for Sustainable Development,” A/RES/70/1, 2015. [Online]. Available: https://sdgs.un.org/2030agenda. Dated: 14/10/2026.

[22] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA, USA: Addison-Wesley, 1994.

[23] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

[24] S. Newman, Building Microservices, 2nd ed. Sebastopol, CA, USA: O’Reilly, 2021.

[25] A. N. Oppenheim, Questionnaire Design, Interviewing and Attitude Measurement. London, U.K.: Pinter, 1992.